arXiv:2511.04427v3 [cs.SE] 26 Jan 2026

Speed at the Cost of Quality

How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects

Hao He
Carnegie Mellon University
Pittsburgh, Pennsylvania, USA

Courtney Miller
Carnegie Mellon University
Pittsburgh, Pennsylvania, USA

Shyam Agarwal
Carnegie Mellon University
Pittsburgh, Pennsylvania, USA

Christian Kästner
Carnegie Mellon University
Pittsburgh, Pennsylvania, USA

Bogdan Vasilescu
Carnegie Mellon University
Pittsburgh, Pennsylvania, USA

Abstract

Large language models (LLMs) have demonstrated the promise to revolutionize the field of software engineering. Among other things, LLM agents are rapidly gaining momentum in software development, with practitioners reporting a multifold increase in productivity after adoption. Yet, empirical evidence is lacking around these claims. In this paper, we estimate the *causal* effect of adopting a widely popular LLM agent assistant, namely Cursor, on *development velocity* and *software quality*. The estimation is enabled by a state-of-the-art difference-in-differences design comparing Cursor-adopting GitHub projects with a matched control group of similar GitHub projects that do not use Cursor. We find that the adoption of Cursor leads to a statistically significant, large, but transient increase in project-level development velocity, along with a substantial and persistent increase in static analysis warnings and code complexity. Further panel generalized-method-of-moments estimation reveals that increases in static analysis warnings and code complexity are major factors driving long-term velocity slowdown. Our study identifies quality assurance as a major bottleneck for early Cursor adopters and calls for it to be a first-class citizen in the design of agentic AI coding tools and AI-driven workflows.

CCS Concepts

• **Software and its engineering** → *Development frameworks and environments*; • **Computing methodologies** → *Intelligent agents*.

ACM Reference Format:
Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. 2026. Speed at the Cost of Quality: How Cursor AI Increases Short-Term Velocity and Long-Term Complexity in Open-Source Projects. In *23rd International Conference on Mining Software Repositories (MSR '26), April 13–14, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 19 pages. https://doi.org/10.1145/3793302.3793349

1 Introduction

Large language models (LLMs) have demonstrated remarkable capabilities in code generation, achieving near-human performance across various software engineering tasks [48, 64]. Among emerging applications, LLM agent assistants—tools that combine LLMs with autonomous capabilities to inspect project files, execute commands, and iteratively develop code—represent a particularly promising direction for integrating LLMs into software development. For example, Cursor [8], a popular LLM agent assistant, has generated considerable enthusiasm among practitioners, with developers self-reporting multi-fold (as large as 10x) productivity increases and claiming transformative workflow impacts [10, 15].

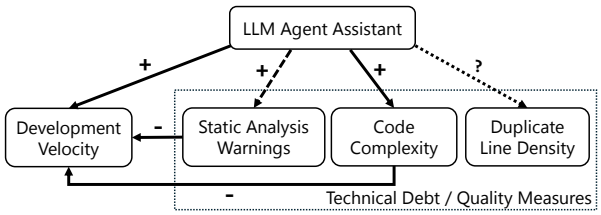

Figure 1: Our theory around how LLM agent assistants impact software development. Solid lines indicate causal relationships supported by our analysis; dashed lines indicate partial evidence; and dotted lines indicate inconclusive evidence.

However, substantial concerns have been raised about the quality of LLM-generated code and the long-term consequences of AI-driven development workflows. Studies documented that AI coding assistants can produce code with security vulnerabilities [93, 95], performance issues [75], code smells [101], and increased complexity [78]. Yet, these findings, derived from evaluations of early Codex models in controlled experiments [95], completion-based tools like early GitHub Copilot [75, 93, 101], or chat-based interfaces like ChatGPT [78], may not generalize to modern LLM agent assistants, which represent a qualitative shift in architecture and integration, not simply incremental improvement. Completion tools suggest individual lines based on immediate context; chat-based assistants require context-switching to formulate queries and integrate responses. Both operate at the periphery of the development workflow, with developers remaining the primary agents and maintaining oversight of AI-generated code at a granular level.

LLM agent assistants like Cursor, by contrast, are tightly integrated into the IDE with persistent codebase awareness, autonomously navigating files, proposing multi-file refactorings, and implementing features spanning dozens of files—all within the development environment. This architectural difference has profound implications that cannot be extrapolated from prior studies: Automation scope shifts from accelerating typing to automating entire

This work is licensed under a Creative Commons Attribution 4.0 International License.
MSR '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2474-9/2026/04
https://doi.org/10.1145/3793302.3793349